南開大學
Nankai University

计算机学院

并行程序设计实验报告

Lab1：体系结构相关编程

——Cache 局部性与超标量优化性能实验

姓名：
学号：
专业：计算机科学与技术
班级：
源码链接：

2026年 4月 2日

1 实验目的与要求

本实验围绕现代处理器性能中两类典型因素展开：一类是由存储层次结构决定的 cache 局部性，另一类是由流水线和发射宽度决定的指令级并行能力。按照课程 PPT 与 Lab1 实验说明，本次作业要求完成两个程序设计任务：其一，计算给定 $n \times n$ 矩阵每一列与给定向量的内积，分别实现逐列访问的平凡算法和 cache 优化算法；其二，计算长度为 n 的数组元素之和，分别实现链式平凡累加和适合超标量处理器的优化算法。在此基础上，报告需要给出程序设计思路、实验平台配置、实验方案设计、实验结果呈现及分析。本文在完成基础要求的同时，进一步对循环展开和不同编译优化等级对性能的影响进行了补充测试。

2 实验平台与实验方案

实验平台为 Intel Core i9-14900HX 处理器，基准频率 2.20 GHz，24 个物理核心，32 个逻辑处理器，内存容量 31.8 GB，操作系统为 Microsoft Windows 11 家庭中文版 64 位，编译器为 TDM-GCC 10.3.0。根据任务管理器与脚本采集结果，处理器的 L1 Cache 为 2.1 MB，L2 Cache 为 32.0 MB，L3 Cache 为 36.0 MB。程序在 VS Code 终端环境下编译与运行，基础测试统一采用 `-O2` 编译选项，补充测试进一步比较了 `-O0`、`-O2` 和 `-O3` 三种优化等级。

测试数据均采用固定规则生成，以保证结果可复现并便于进行正确性校验。程序启动时先进行正确性检查，终端输出为 `Correctness check passed.`，说明各实现版本在给定数据下得到了一致结果。由于小规模问题单次执行时间较短，测试程序会根据规模自动调整重复次数，再以平均执行时间作为最终结果，以减小计时误差。矩阵内积实验选取 $n = 64, 128, 256, 384, 512, 768, 1024, 1536, 2048$ 九组规模；求和实验选取 $n = 1024$ 至 16777216 的八组规模。除基础测试外，矩阵实验还增加了 cache 版本的四路循环展开实现，求和实验增加了双链路四路展开和两两归约实现，用于补充分析不同优化手段的作用。

3 实验一：矩阵每列与向量内积

3.1 程序设计思路

设矩阵为 $B \in \mathbb{R}^{n \times n}$，向量为 $a \in \mathbb{R}^n$，结果向量为 $sum \in \mathbb{R}^n$，则第 i 个输出满足

$$sum_i = \sum_{j=0}^{n-1} B_{j,i} \cdot a_j.$$

平凡算法直接按照“按列求内积”的方式实现：外层循环固定列号 i，内层循环遍历所有行号 j，依次累加 $B_{j,i} \cdot a_j$。该算法逻辑直观，但在 C 语言的行主存储下，对矩阵元素的访问步长为 n，会频繁跨越不同 cache line，空间局部性较差。

cache 优化算法保持相同的数学计算量，只调整循环顺序：先按行遍历矩阵，再将当前行对各列结果的贡献一次性累加到结果向量中。这样内层循环访问的是连续存储的一整行数据，能够更充分地利用 cache line 中的相邻元素，从而减少无效访存和缓存失效带来的性能损失。进一步地，在此基础上又实现了四路循环展开版本，每次循环连续处理四个元素，以减少循环控制开销，并为编译器提供更多指令重排空间。

3.2 实验结果与分析

表 1 给出了在 `-O2` 条件下平凡算法和 cache 优化算法的基础测试结果。

表 1: 矩阵每列与向量内积基础测试结果

n	naive_ms	cache_ms	speedup	repeats_naive	repeats_cache
64	0.000903	0.001141	0.791419	292960	292960
128	0.006789	0.004073	1.667030	36616	73232
256	0.060795	0.031533	1.927994	4576	9152
384	0.129354	0.079284	1.631521	2032	4064
512	0.254104	0.115108	2.207516	1144	2288
768	0.700431	0.277682	2.522421	508	1016
1024	3.335070	0.500330	6.665738	71	568
1536	7.004510	1.146008	6.112097	31	248
2048	37.766971	3.649994	10.347132	17	68

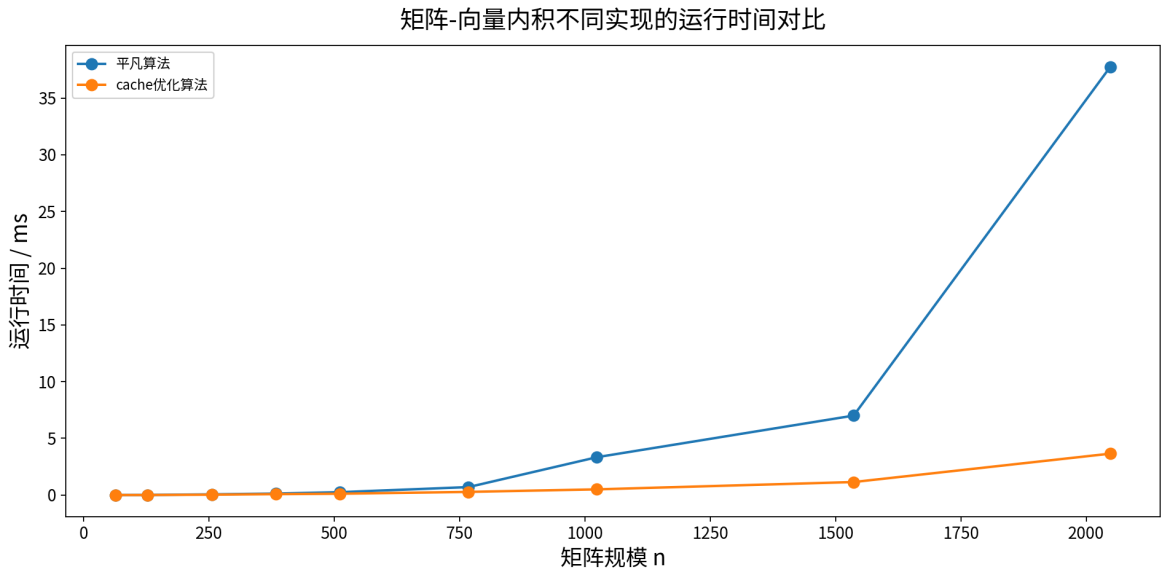

图 1: 不同矩阵规模下平凡算法与 cache 优化算法的运行时间对比

基础结果表明，随着矩阵规模增大，cache 优化算法的优势迅速增大。$n = 512$ 时加速比已经达到 2.21 倍；当 $n = 1024$ 时加速比提升到 6.67 倍；到 $n = 2048$ 时进一步达到 10.35 倍。说明该实验的主要瓶颈并不在乘加次数，而在访存模式。平凡算法按列访问矩阵，在行主存储布局下会不断跨越不同 cache line，缓存利用率较低；而优化算法按行连续读取矩阵元素，明显改善了空间局部性，因此在大规模下具有更高吞吐能力。这一现象与本机 2.1 MB 的 L1 Cache、32.0 MB 的 L2 Cache 和 36.0 MB 的 L3 Cache 容量相一致：随着矩阵规模不断扩大，数据逐渐超出低层缓存的有效覆盖范围，访存模式对性能的影响被明显放大。

为考察进一步优化的作用，又对循环展开和编译优化等级进行了补充测试。表 2 给出了循环展开的代表性结果，表 3 给出了不同编译优化等级下的代表性结果。

表 2: 矩阵内积中循环展开的代表性结果（`-O2`）

n	cache_ms	unroll4_ms	spd._cache	spd._unroll4
256	0.031533	0.024789	1.93	2.45
1024	0.500330	0.489865	6.67	6.81
2048	3.649994	3.443459	10.35	10.97

表 3: 矩阵内积中不同编译优化等级的代表性结果（$n = 2048$）

版本	`-O0`	`-O2`	`-O3`
naive	61.1137	37.7670	33.7209
cache	9.0845	3.6500	1.2507

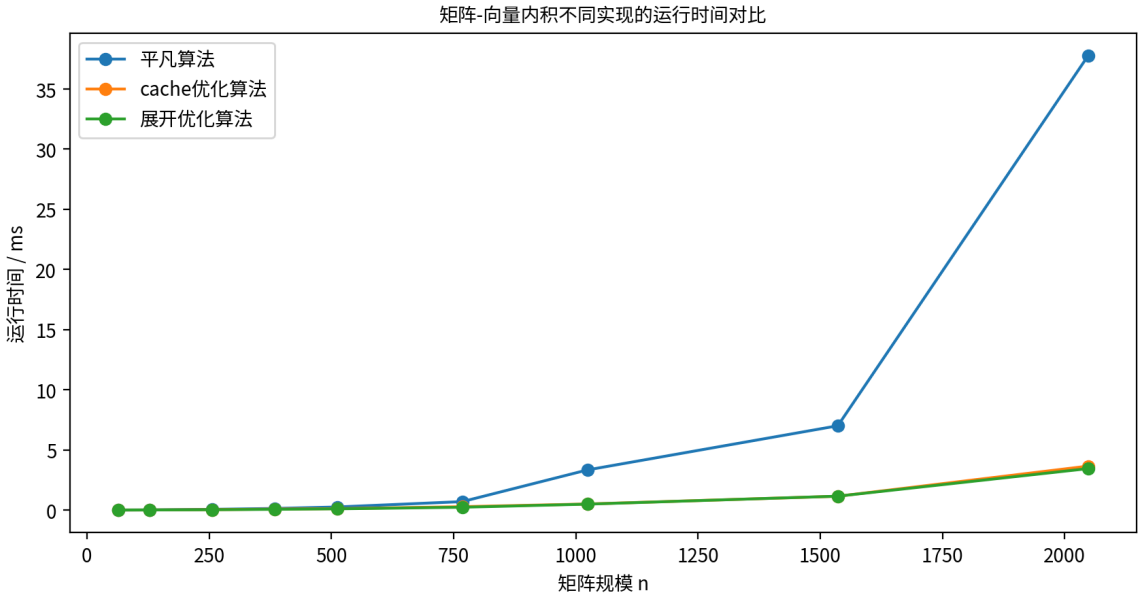

图 2: 矩阵内积实验中不同实现版本的运行时间对比

补充结果显示，在已经完成 cache 优化的前提下，四路循环展开仍能带来进一步收益，但收益幅度明显小于“按列访问改为按行访问”这一主优化。以 $n = 2048$ 为例，cache 版本相对平凡算法加速 10.35 倍，而展开版本提升到 10.97 倍，说明循环展开主要通过减少循环判断和归纳变量更新带来次一级优化收益。在编译器优化等级方面，`-O0`、`-O2`、`-O3` 的差异较为显著：对 $n = 2048$ 的 cache 版本而言，时间从 9.0845 ms 下降到 3.6500 ms，再进一步下降到 1.2507 ms，表明编译器的指令调度、常量传播、循环优化和向量化能力同样会显著影响最终性能。总体上，矩阵内积实验中最关键的仍是访存顺序优化，循环展开和编译器优化属于在此基础上的进一步提升。

4 实验二：n 个数求和

4.1 程序设计思路

第二个实验要求计算长度为 n 的数组元素之和。平凡算法使用单个累加器，从头到尾逐个执行 `sum += a[i]`。这种写法实现简单，但每一步运算都依赖上一步的结果，会形成一条较长的数据相关链，从而限制处理器流水线和超标量发射单元的并行能力。

优化算法采用两路链式累加的思路：使用两个独立累加器分别处理不同位置的数组元素，最后再把两个部分和相加。这样能够将一条很长的依赖链拆分为两条较短且彼此独立的依赖链，从而更好地发挥处理器的指令级并行能力。在此基础上，又实现了四路展开版本 `dual_unroll4`，进一步减少循环控制开销；同时实现了 `pairwise` 两两归约版本，用于观察树形归约结构在串行 CPU 上的实际表现。

4.2 实验结果与分析

表 4 给出了在 `-O2` 条件下链式平凡算法和两路链式累加算法的基础测试结果。

表 4: n 个数求和基础测试结果

n	naive_ms	dual_ms	speedup	repeats_naive	repeats_dual
1024	0.000875	0.000533	1.639727	390624	390624
4096	0.004069	0.002179	1.867015	97656	195312
16384	0.016338	0.007393	2.209832	24414	48828
65536	0.059309	0.032988	1.797910	6102	6102
262144	0.235940	0.125555	1.879175	1524	3048
1048576	1.124038	0.594254	1.891511	190	380
4194304	4.968629	3.002182	1.655006	94	94
16777216	19.795482	19.021014	1.040716	11	22

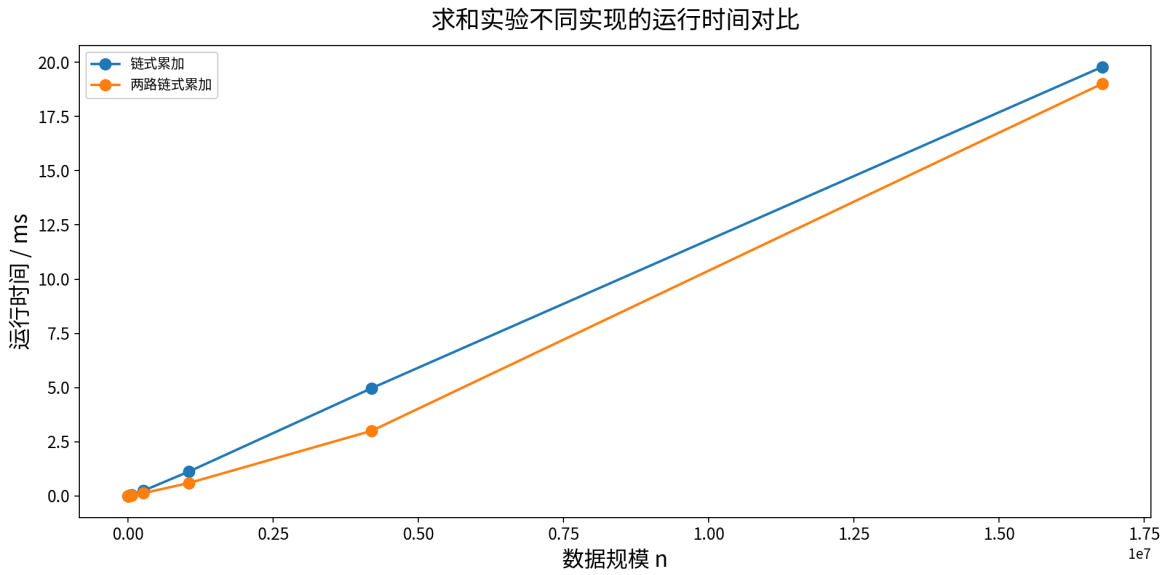

图 3: 不同数据规模下链式累加与两路链式累加的运行时间对比

基础测试表明，两路链式累加在大多数规模下都优于平凡算法。较典型的是 $n = 16384$ 时加速比达到 2.21 倍，在 $n = 262144$ 和 $n = 1048576$ 时分别为 1.88 倍和 1.89 倍，说明将单累加器改为双累加器后，确实能够缩短数据依赖链并提高指令级并行性。不过当规模增大到 $n = 16777216$ 时，加速比下降到 1.04 倍，说明在超大规模下，性能瓶颈已不再主要来自算术依赖，而更多受到内存带宽和整体数据搬移开销的影响。

进一步地，表 5 和表 6 分别给出了补充算法与不同编译优化等级的代表性结果。

表 5: n 个数求和补充算法的代表性结果（`-O2`）

n	dual_ms	unroll4_ms	pairwise	spd._dual	spd._unroll4
16384	0.007393	0.004413	0.011743	2.21	3.70
1048576	0.594254	0.310325	2.339208	1.89	3.62
16777216	19.021014	16.754614	59.875982	1.04	1.18

表 6: n 个数求和中不同编译优化等级的代表性结果（$n = 1048576$）

版本	`-O0`	`-O2`	`-O3`
naive	3.5348	1.1240	0.4229
dual	2.0413	0.5943	0.2110

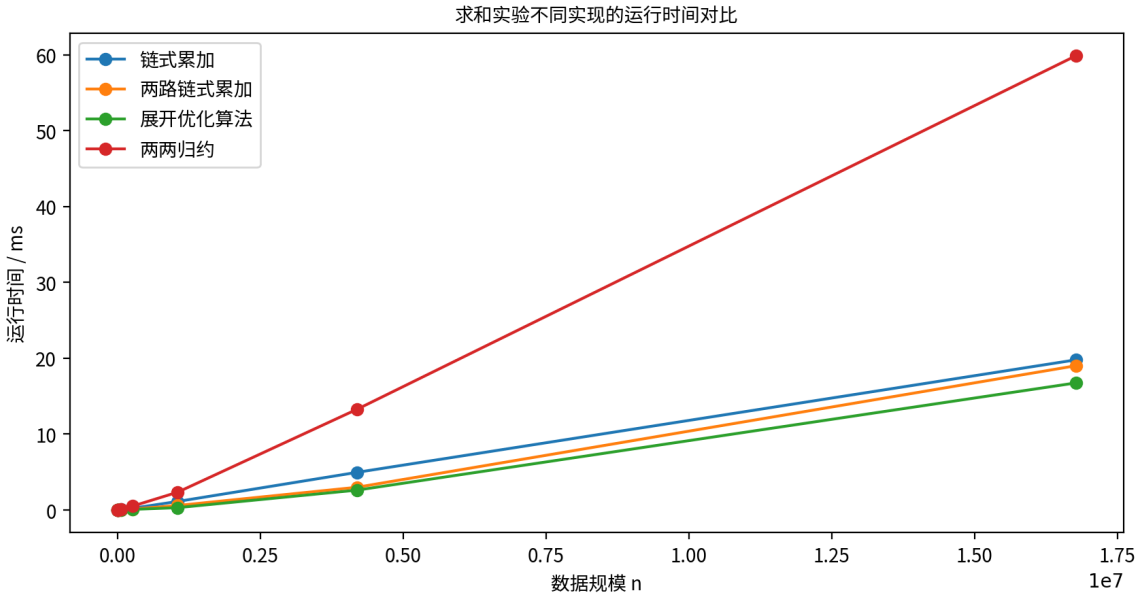

图 4: 求和实验中不同实现版本的运行时间对比

补充结果表明，四路展开版本 `dual_unroll4` 的表现明显优于基础的 `dual` 版本。以 $n = 1048576$ 为例，`dual` 用时 0.5943 ms，而展开版本下降到 0.3103 ms，相对平凡算法的加速比提升到 3.62 倍，说明“减少依赖链”和“减少循环控制开销”能够形成叠加收益。相比之下，`pairwise` 在串行实现中的表现并不理想：同样在 $n = 1048576$ 时，用时达到 2.3392 ms；在 $n = 16777216$ 时进一步上升到 59.8760 ms，明显慢于链式与展开版本。这说明树形归约虽然在并行算法设计中常见，但在当前串行 CPU 上，多轮写回与额外数据移动会抵消其理论优势。

编译器优化等级对求和实验的影响同样显著。以 $n = 1048576$ 为例，平凡版本在 `-O0`、`-O2`、`-O3` 下分别为 3.5348 ms、1.1240 ms 和 0.4229 ms；双链路版本分别为 2.0413 ms、0.5943 ms 和 0.2110 ms。说明在求和这类算术密集型且结构较规则的程序中，编译器优化能够充分发挥作用。当问题规模进一步增大到 $n = 16777216$ 时，`-O3` 仍优于 `-O2` 和 `-O0`，但算法之间的差距相对缩小，再次说明大规模情形下访存开销会逐渐削弱单纯依赖算术结构优化所带来的收益。

5 实验总结

本次实验通过两个典型程序设计任务考察了 cache 局部性与超标量优化对程序性能的影响。矩阵每列与向量内积实验表明，改变访存顺序、提高 cache 命中率是提升大规模程序性能的关键；在此基础上，循环展开和编译器优化还能带来进一步收益，但作用层级低于主访存优化。n 个数求和实验表明，双链路累加与展开优化能够明显缩短指令依赖链，提高指令级并行效率；但当数据规模极大时，内存访问与带宽约束又会重新成为主要瓶颈。总体来看，本实验验证了现代处理器性能并不只取决于算法的算术复杂度，还与数据访问模式、依赖关系组织方式以及编译器优化能力密切相关。